AX.25 Terminal for iOS

Implementation Specification

Version 1.0 — March 2026

DRAFT — For Implementation Use

A complete specification for building a native iOS application that provides full AX.25 connected-mode terminal access to packet radio networks, compatible with Bluetooth LE KISS TNCs, TCP KISS TNCs, and built-in AFSK software modem via audio interfaces.

Table of Contents

1. Executive Summary
2. System Architecture Overview
3. Transport Layer: TNC Communication
   3.1 BLE KISS TNC Interface
   3.2 TCP KISS TNC Interface
   3.3 Built-in AFSK Software Modem
   3.4 KISS Protocol Implementation
4. AX.25 Data Link Layer
   4.1 Frame Format
   4.2 Address Field Encoding
   4.3 Control Field and Frame Types
   4.4 Connected Mode State Machine
   4.5 Connectionless (UI) Mode
   4.6 Timers and Counters
   4.7 Parameter Negotiation (XID)
5. Terminal Emulation Layer
   5.1 Terminal Display
   5.2 Input Handling
   5.3 YAPP File Transfer Protocol
6. Monitor Mode
7. Node and Connection Management
   7.1 Node Database
   7.2 Digipeater Routing
   7.3 Location-Aware Smart Paths
8. Data Persistence and Sync
9. Radio Integration
10. User Interface Specification
11. Error Handling and Edge Cases
12. Testing Strategy
13. Appendix A: KISS Protocol Byte Reference
14. Appendix B: AX.25 Frame Type Reference
15. Appendix C: BLE KISS UUID Reference
16. Appendix D: AFSK Modem DSP Parameters

1. Executive Summary

This document specifies an iOS application that provides a full-featured AX.25 packet radio terminal. The app enables licensed amateur radio operators to connect to packet nodes, bulletin board systems (BBS), and other AX.25 stations using an iPhone or iPad. It communicates with a radio transceiver through an external Terminal Node Controller (TNC) via Bluetooth Low Energy or TCP/IP, or through a built-in software AFSK modem via a USB audio interface.

The application implements the AX.25 v2.0 connected-mode data link layer protocol sufficient for interactive terminal sessions, over-the-air traffic monitoring, and binary file transfer via the YAPP protocol. It does not implement full AX.25 v2.2 extended mode (modulo-128 sequencing), as the vast majority of packet nodes and BBS systems in active use operate with v2.0 modulo-8 framing. However, the architecture is designed to allow v2.2 support to be added incrementally.

1.1 Scope

The application covers the following functional areas:

- KISS TNC communication over BLE (Bluetooth Low Energy) and TCP/IP
- Built-in 1200 baud AFSK (Bell 202) software modem for use with USB audio interfaces
- AX.25 v2.0 connected-mode protocol state machine (SABM/UA/DISC/DM/I/RR/RNR/REJ/FRMR/UI)
- Full-screen terminal emulator with retro color themes
- Over-the-air AX.25 frame monitor with filtering and search
- YAPP binary file transfer protocol (send and receive)
- Node database with iCloud sync
- Digipeater path management with location-aware smart routing
- Automatic TNC discovery via BLE scanning and DNS-SD (Bonjour)
- Frequency control on supported radios via vendor BLE commands

1.2 Target Platform

- iOS 18.2 or later
- iPhone and iPad (Universal app)
- Swift 6, SwiftUI, Combine, CoreBluetooth, AVFoundation, Accelerate

1.3 Reference Documents

- AX.25 Link Access Protocol for Amateur Packet Radio, Version 2.2, July 1998 (NJ7P, N7LEM, N7OO)
- AX.25 Link Access Protocol for Amateur Packet Radio, Version 2.0, October 1984 (WB4JFI)
- KISS TNC Protocol Specification (KA9Q, K3MC, WB6RQN, August 1986)
- Specification for KISS over BLE (OH7LZB / hessu, aprs-specs repository)
- Specification for DNS-SD discovery of KISS over TCP (OH7LZB / hessu, aprs-specs repository)
- YAPP File Transfer Protocol (WA7MBL, Revision 1.1, June 1986)
- YAPP Extensions (F6FBB/FC1EBN, TPK 1.65a / FBB 5.14)
- Bell 202 AFSK Modulation Standard

2. System Architecture Overview

The application is organized into four primary layers, each with clear responsibilities and well-defined interfaces between them.

2.1 Layer Diagram

From top to bottom:

UI Layer: SwiftUI Views (Terminal, Monitor, Settings, Node List)
Session Layer: AX25Session (connect, send, receive, disconnect)
Data Link Layer: AX25Engine (frame encode/decode, state machine, timers)
Transport Layer: KISSTransport (BLE / TCP / Software Modem)

2.2 Key Protocols

The transport layer communicates with the TNC using the KISS protocol. The TNC handles the physical layer (HDLC framing, NRZI encoding, AFSK modulation, PTT control, carrier sense). When the built-in software modem is used, the app itself performs AFSK modulation/demodulation and the audio interface handles PTT via VOX or a dedicated tone.

The data link layer implements AX.25 frame encoding/decoding and the connected-mode state machine. It constructs SABM, DISC, UA, DM, I, RR, RNR, REJ, FRMR, and UI frames, manages send/receive sequence numbers, and handles retransmission timers.

The session layer provides a clean async Swift API for the UI: connect(to:via:), send(data:), disconnect(), and publishes received data, connection state changes, and error events via Combine publishers.

2.3 Swift Module Structure

```
PacketTerminal/
  App/                    # SwiftUI App entry, navigation
  UI/                     # SwiftUI views
    TerminalView.swift
    MonitorView.swift
    NodeListView.swift
    ConnectionSheet.swift
    SettingsView.swift
  Session/                # High-level session management
    AX25Session.swift
    YAPPTransfer.swift
  AX25/                   # Protocol engine
    AX25Frame.swift
    AX25Engine.swift
    AX25StateMachine.swift
    AX25Address.swift
  KISS/                   # KISS framing
    KISSCodec.swift
    KISSTransport.swift   # Protocol interface
```

```
Transport/              # Physical transports
  BLEKISSTransport.swift
  TCPKISSTransport.swift
  AFSKModemTransport.swift
Modem/                  # Software AFSK modem
  AFSKModulator.swift
  AFSKDemodulator.swift
  PTTController.swift
Model/                  # Data models
  Node.swift
  Digipeater.swift
  MonitorFrame.swift
Persistence/            # CoreData + CloudKit
  NodeStore.swift
```

3. Transport Layer: TNC Communication

The transport layer abstracts all communication with external TNC hardware (or the internal software modem) behind a common protocol. All transports accept and produce raw KISS frames.

3.1 KISSTransport Protocol

Every transport conforms to this Swift protocol:

```
protocol KISSTransport: AnyObject {
    var name: String { get }
    var isConnected: Bool { get }
    var onFrameReceived: ((Data) -> Void)? { get set }
    var onConnectionStateChanged: ((Bool) -> Void)? { get set }
    func connect() async throws
    func disconnect()
    func send(kissFrame: Data) async throws
}
```

The onFrameReceived callback delivers complete, un-escaped KISS frames (including the type byte). The KISSCodec layer handles SLIP framing (escape/unescape) and frame assembly from the raw byte stream.

3.2 BLE KISS TNC Interface

3.2.1 Discovery

The app scans for BLE peripherals advertising the KISS TNC Service UUID. This UUID was defined by OH7LZB in the BLE KISS specification and is used by the Mobilinkd TNC3, TNC4, and other compatible devices:

```
KTS_SERVICE_UUID = 00000001-ba2a-46c9-ae49-01b0961f68bb
```

The scan is initiated from the Settings screen. Discovered peripherals are displayed in a list showing the BLE peripheral name (e.g., "TNC4 Mobilinkd", "Dire Wolf TNC"). The user taps a device to select it as the active TNC.

3.2.2 Connection

After the user selects a BLE peripheral:

1. Call CBCentralManager.connect() on the peripheral.
2. On successful connection, discover services matching KTS_SERVICE_UUID.
3. Discover characteristics within that service.
4. Identify the TX characteristic (KTS_TX_CHAR_UUID = 00000002-ba2a-46c9-ae49-01b0961f68bb) and RX characteristic (KTS_RX_CHAR_UUID = 00000003-ba2a-46c9-ae49-01b0961f68bb).
5. Subscribe to notifications on the RX characteristic.
6. Store a reference to the TX characteristic for sending.

Important: The app must NOT pair through iOS Bluetooth Settings. BLE KISS TNCs are discovered and connected entirely through CoreBluetooth within the app. Directing users to iOS Settings will cause connection failures.

3.2.3 Data Transfer

Receiving: When the TNC has an AX.25 frame to deliver, it KISS-encodes it and writes to the RX characteristic, triggering a BLE notification. The app receives these bytes in peripheral(_:didUpdateValueFor:error:). Because BLE has a maximum transfer unit (MTU, typically 20–512 bytes negotiated at connection time), a single KISS frame may span multiple BLE notifications. The app must buffer incoming bytes and parse KISS frame boundaries (0xC0 FEND delimiters) from the assembled stream.

Transmitting: The app KISS-encodes the AX.25 frame and writes it to the TX characteristic using writeValue(_:for:type:) with .withoutResponse. If the encoded frame exceeds the negotiated MTU, it must be split into MTU-sized chunks and written sequentially. Each chunk is a raw byte segment; the TNC reassembles them. The FEND delimiters within the KISS encoding serve as the logical frame boundaries.

3.2.4 MTU Negotiation

After connecting, the app should observe peripheral(_:didReadRSSI:error:) and the maximumWriteValueLength(for:) property to determine the effective MTU. Modern BLE 4.2+ devices (including TNC3/TNC4) support MTUs of 185 bytes or more. The app must handle the minimum BLE MTU of 20 bytes gracefully.

3.3 TCP KISS TNC Interface

3.3.1 DNS-SD Discovery

The app uses NWBrowser (Network framework) to discover KISS TNCs advertised via DNS-SD (Bonjour) with service type _kiss-tnc._tcp. Dire Wolf (version 1.7+) advertises itself this way. Discovered services are shown alongside BLE TNCs in the TNC selection list.

The TXT record may contain a "pn" key with port/frequency information (e.g., "pn=144.800 1200,434.125 9600"). The app should parse and display this if present.

3.3.2 Manual Configuration

The user may also manually enter a hostname/IP and TCP port for a KISS TNC not discoverable via DNS-SD.

3.3.3 Connection

Use NWConnection (Network framework) to establish a TCP connection to the resolved host and port. The KISS protocol runs directly over the TCP stream with no additional framing beyond KISS itself. Multiple KISS frames may arrive in a single TCP read, and a single frame may be split across reads. The same KISS frame assembly logic used for BLE applies here.

3.4 Built-in AFSK Software Modem

When no external TNC is available, the app can use a USB audio interface (such as Digirig Lite or AIOC) to modulate and demodulate 1200 baud AFSK audio directly, functioning as its own TNC. This section specifies the modem in complete detail.

3.4.1 Audio Interface Requirements

iOS does not allow apps to access USB audio class devices in the same way as macOS. However, USB audio interfaces that present as standard Core Audio devices are accessible via AVAudioEngine. The supported interfaces include:

- Digirig Lite: Recognized as a USB audio device. PTT is controlled via a dedicated GPIO line triggered by audio output presence.
- Digirig VOX PTT Cable: PTT is triggered by a sub-audible tone on the audio output channel.

- AIOC (All-In-One Cable): Firmware v1.4.1+. Recognized as USB audio. AutoPTT mode uses VOX-like detection.

3.4.2 AFSK Modulator (TX)

Modulation scheme: Bell 202 AFSK. Mark (binary 1) = 1200 Hz. Space (binary 0) = 2200 Hz.

Baud rate: 1200 baud (833.33 µs per symbol).

Sample rate: 22050 Hz (exactly 18.375 samples per symbol). Alternatively 44100 Hz (36.75 samples per symbol) if needed for the audio device.

Phase continuity: Critical. The modulator must maintain continuous phase across symbol transitions. This means the sine wave generator does not reset its phase angle when switching between mark and space frequencies. Implement as a numerically controlled oscillator (NCO): maintain a running phase accumulator and add the per-sample phase increment for the current frequency.

```
phase += (currentFreq / sampleRate) * 2π
sample = sin(phase)
```

Pre-emphasis: Optional. A 6 dB/octave high-frequency boost can improve decode rates on FM radios that apply de-emphasis. Implement as a first-order high-pass filter with a cutoff around 1200 Hz applied to the AFSK output. Make this configurable.

TX pipeline:

1. Receive raw AX.25 frame bytes from the AX25Engine.
2. Perform HDLC framing: prepend flags (0x7E), apply zero-bit stuffing, compute and append FCS (CRC-CCITT), append closing flag.
3. Apply NRZI encoding: a 0 bit causes a transition; a 1 bit causes no transition.
4. Feed the NRZI bit stream into the AFSK modulator NCO.
5. Send the resulting audio samples to the output audio device via AVAudioEngine.
6. Key PTT before audio begins (TX delay), unkey after audio ends (TX tail).

3.4.3 AFSK Demodulator (RX)

Method: Non-coherent correlation demodulator. For each sample window (one symbol period), compute the correlation of the received signal with 1200 Hz and 2200 Hz reference signals. The frequency with the higher correlation magnitude is the detected symbol.

Implementation using the Accelerate framework for DSP performance:

1. Capture audio from the input device via AVAudioEngine input tap.
2. Apply an optional bandpass filter (900–2500 Hz) to reject out-of-band noise.
3. For each symbol period of samples, compute the dot product with pre-computed sine/cosine tables for both 1200 Hz and 2200 Hz. The magnitude of each is $\text{sqrt}(I^2 + Q^2)$.
4. The bit value is 1 (mark) if the 1200 Hz magnitude exceeds the 2200 Hz magnitude, else 0 (space).
5. Apply NRZI decoding: compare each bit to the previous. Same = 1, different = 0.
6. Feed the resulting bit stream into the HDLC decoder: detect flag sequences (0x7E), remove zero-bit stuffing, extract frame bytes, verify FCS.
7. Deliver valid frames to the AX25Engine.

3.4.4 HDLC Framing (for Software Modem)

This is only needed when the software modem is active. When using an external KISS TNC, the TNC handles HDLC internally.

Flag: 0x7E (01111110). Sent continuously as preamble and between frames.

Zero-bit stuffing: After any sequence of five consecutive 1-bits in the data, insert a 0-bit. On receive, after five consecutive 1-bits, discard the following 0-bit.

FCS: CRC-CCITT (polynomial 0x8408, reflected). Computed over all bytes between (but not including) the opening and closing flags. Transmitted LSB first. The receiver computes the CRC over the data plus the received FCS; a valid frame yields a residual of 0xF0B8.

NRZI encoding: 0-bit = transition (change in tone). 1-bit = no transition (same tone). This is applied after zero-bit stuffing on transmit and reversed before zero-bit removal on receive.

Bit ordering: LSB first for all bytes.

3.4.5 PTT Control

PTT (push-to-talk) control varies by audio interface:

- **VOX-based (Digirig VOX, AIOC AutoPTT):** The audio output signal itself triggers PTT. The app generates a short burst of a single tone (e.g., 1000 Hz) for the TX delay period before sending AFSK data, and a brief TX tail tone after the last frame. The tone frequency should be configurable (default 1000 Hz; some radios need 12 kHz to avoid bleed-through).
- **Digirig Lite:** PTT is controlled via a specific audio output pattern recognized by the Digirig hardware. Consult the Digirig Lite documentation for the specific signaling.

3.4.6 Channel Access (CSMA)

When the software modem is active, the app must implement p-persistence CSMA for channel access:

1. Before transmitting, check if the channel is clear by monitoring the demodulator output for carrier (DCD).
2. If the channel is clear, generate a random number 0–255. If it is less than P (persistence, default 63 = 25%), transmit. Otherwise, wait one slot time (default 100 ms) and repeat.
3. If the channel is busy, wait until it becomes clear, then apply the p-persistence algorithm.

3.5 KISS Protocol Implementation

The KISS protocol uses SLIP-style framing to exchange AX.25 frames between the host and TNC.

3.5.1 Special Bytes

Name	Value	Description
FEND	`0xC0`	Frame End delimiter
FESC	`0xDB`	Frame Escape
TFEND	`0xDC`	Transposed Frame End
TFESC	`0xDD`	Transposed Frame Escape

3.5.2 Frame Structure

A KISS frame on the wire looks like:

```
FEND | Type Byte | Payload (escaped) | FEND
```

The type byte encodes the port number (high nibble, 0–F) and command code (low nibble). For a single-port TNC, the port is always 0.

3.5.3 Command Codes (Host to TNC)

Code	Name	Description
`0x00`	Data Frame	Payload is an AX.25 frame to transmit

`0x01`	TX Delay	Transmitter keyup delay in 10 ms units (default 50 = 500 ms)
`0x02`	Persistence (P)	P value 0–255, maps to p = (P+1)/256 (default 63)
`0x03`	Slot Time	CSMA slot interval in 10 ms units (default 10 = 100 ms)
`0x04`	TX Tail	Time to hold TX after frame in 10 ms units (default 0)
`0x05`	Full Duplex	0 = half duplex (default), nonzero = full duplex
`0x06`	Set Hardware	Vendor-specific. Used by Mobilinkd for TNC configuration.
`0xFF`	Return	Exit KISS mode (not used by most modern TNCs)

3.5.4 Escape Encoding

When the payload contains 0xC0 (FEND), it is replaced with the two-byte sequence 0xDB 0xDC (FESC TFEND). When the payload contains 0xDB (FESC), it is replaced with 0xDB 0xDD (FESC TFESC). On receive, the inverse transformation is applied.

3.5.5 Frame Assembly

The KISS codec maintains a byte buffer. Incoming bytes from the transport (BLE notifications, TCP reads) are appended. The codec scans for FEND delimiters:

1. Discard bytes until a FEND is seen (sync).
2. Accumulate bytes after the FEND, applying un-escaping.
3. When a second FEND is seen, the accumulated bytes form a complete KISS frame (type byte + payload).
4. Back-to-back FENDs (empty frames) are silently discarded.
5. Deliver the payload (with type byte stripped) to the AX25Engine.

4. AX.25 Data Link Layer

The AX.25 engine is the core of the application. It encodes and decodes AX.25 frames, manages the connected-mode state machine, and handles retransmission logic.

4.1 Frame Format

An AX.25 frame, as delivered via KISS (i.e., without HDLC flags, bit stuffing, or FCS—those are handled by the TNC or software modem), has the following structure:

Field	Length	Description
Destination Address	7 bytes	Callsign (6) + SSID byte (1)
Source Address	7 bytes	Callsign (6) + SSID byte (1)
Digipeater Addresses	0–56 bytes	0 to 8 digipeater addresses, 7 bytes each
Control	1 byte	Frame type, P/F bit, sequence numbers
PID	0 or 1 byte	Protocol ID. Present in I and UI frames only.
Information	0–256 bytes	User data. Present in I and UI frames only.

4.2 Address Field Encoding

Each address subfield is 7 bytes. The first 6 bytes encode the callsign in uppercase ASCII, left-justified and padded with spaces, with each byte shifted left by 1 bit. The 7th byte (SSID byte) contains:

- Bit 0 (LSB): Extension bit. 0 = more address bytes follow; 1 = this is the last address in the header.
- Bits 1–4: SSID value (0–15).
- Bit 5: Reserved, set to 1.
- Bit 6: Reserved, set to 1 (but cleared to 0 in source SSID byte to indicate modulo-128 connection, per v2.2 convention for monitoring).
- Bit 7: Command/Response bit. In the destination address: 1 = command, 0 = response. In the source address: 0 = command, 1 = response. Both 1 or both 0 = legacy/old protocol.

For digipeater addresses, bit 7 of the SSID byte is the "Has Been Repeated" (H) bit, set to 1 after a digipeater relays the frame.

4.2.1 Encoding Example

To encode callsign "W2ASM" with SSID 0 as a source address (last address, command frame):

```
'W' << 1 = 0xAE
'2' << 1 = 0x64
'A' << 1 = 0x82
'S' << 1 = 0xA6
'M' << 1 = 0x9A
' ' << 1 = 0x40  (padding)
SSID byte = 0b_0_1_1_0000_1 = 0x61 (extension=1, SSID=0, reserved=11, C/R=0)
```

4.3 Control Field and Frame Types

AX.25 v2.0 uses modulo-8 sequence numbers (3 bits). The control byte encodes the frame type:

4.3.1 I-Frame (Information)

```
Bit pattern: N(R) P N(S) 0
Bits:        7 6 5 4 3 2 1 0
```

N(S) = send sequence number (bits 1–3). N(R) = receive sequence number (bits 5–7). P = poll bit (bit 4). Bit 0 = 0 identifies this as an I-frame.

4.3.2 S-Frame (Supervisory)

```
Bit pattern: N(R) P/F S S 0 1
Bits:        7 6 5  4  3 2 1 0
```

SS Bits	Type	Meaning
00	RR	Receive Ready: ACK through N(R)-1, ready for more
01	RNR	Receive Not Ready: ACK through N(R)-1, temporarily busy
10	REJ	Reject: request retransmission from N(R) onward
11	SREJ	Selective Reject (v2.2 only): retransmit only frame N(R)

4.3.3 U-Frame (Unnumbered)

```
Bit pattern: M M M P/F M M 1 1
```

MMMMM	Type	Meaning
00101	SABM	Set Async Balanced Mode (connect, v2.0)
01101	SABME	Set ABM Extended (connect, v2.2 modulo-128)
01000	DISC	Disconnect
00110	UA	Unnumbered Acknowledge
00011	DM	Disconnected Mode (refusal or status)
00000	UI	Unnumbered Information (connectionless data)
10000	FRMR	Frame Reject (unrecoverable error)
10111	XID	Exchange Identification (parameter negotiation, v2.2)
11100	TEST	Test frame

4.4 Connected Mode State Machine

The state machine governs the lifecycle of a single AX.25 connection. It is modeled directly on the AX.25 v2.0 specification state table with practical simplifications for a terminal application.

4.4.1 States

State	Name	Description
S1	Disconnected	No connection. Respond to SABM with UA or DM. Ignore all else.
S2	Awaiting Connection	SABM sent, waiting for UA or DM. T1 running.
S3	Connected	Active data transfer. I-frames sent/received.
S4	Timer Recovery	T1 expired during data transfer. Sent RR with P=1, awaiting response.
S5	Awaiting Disconnect	DISC sent, waiting for UA or DM. T1 running.

4.4.2 Connection Establishment

When the user initiates a connection to a remote station:

7. Construct a SABM command frame with P=1, addressed to the destination callsign-SSID, optionally via digipeater(s).
8. Send the frame via KISS.
9. Enter state S2 (Awaiting Connection). Start timer T1. Set retry counter RC = 0.
10. If a UA response is received, enter state S3 (Connected). Stop T1. Reset V(S), V(R), V(A) to 0.
11. If a DM response is received, enter state S1 (Disconnected). Notify user: connection refused.
12. If T1 expires, increment RC. If RC < N2 (retry limit, default 10), retransmit SABM and restart T1. If RC >= N2, enter S1 and notify user: connection timed out.

4.4.3 Data Transfer (State S3)

Sending data: When the user sends text, the session layer segments it into chunks no larger than the negotiated PACLEN (default 256 bytes for v2.0). Each chunk becomes the information field of an I-frame. The I-frame is constructed with N(S) = V(S), N(R) = V(R), then V(S) is incremented modulo 8. The frame is queued for transmission. Up to K (MAXFRAME, default 4 for v2.0, max 7) outstanding unacknowledged I-frames may be sent. If the window is full, further sends are queued until acknowledgment.

Receiving data: When an I-frame is received with N(S) == V(R), the information field is delivered to the session layer (and then to the terminal display). V(R) is incremented modulo 8. An RR response with N(R) = V(R) is sent to acknowledge. If N(S) != V(R), the frame is out of sequence; send a REJ with N(R) = V(R) to request retransmission.

Piggybacked ACKs: Every I-frame sent includes the current V(R) in its N(R) field, implicitly acknowledging all frames up to V(R)-1. The engine should piggyback acknowledgments rather than sending separate RR frames whenever possible, to reduce channel usage.

Acknowledgment processing: When any frame is received with N(R), all I-frames with sequence numbers up to N(R)-1 are acknowledged. Remove them from the retransmission buffer. Update V(A) = N(R).

4.4.4 Disconnection

User-initiated: Send a DISC command with P=1. Enter state S5. Start T1. On UA received, enter S1. On T1 timeout, retry up to N2 times, then enter S1.

Remote-initiated: If a DISC is received in state S3, respond with UA, enter S1, and notify the user.

Abnormal: If a DM is received in S3, or if FRMR is received, or if T1 expires N2 times during timer recovery (S4), the link is considered failed. Enter S1 and notify the user.

4.5 Connectionless (UI) Mode

UI frames are used for unconnected, broadcast communication. They are primarily seen in APRS, beacons, and node identification. The monitor mode displays all UI frames. The PID field in UI frames is typically 0xF0 (no layer 3). The app does not need to send UI frames for terminal operation but must decode them for the monitor.

4.6 Timers and Counters

Timer	Name	Default	Purpose
T1	Acknowledgment Timer	3000 ms	Retransmit if no ACK
T2	Response Delay Timer	100 ms	Delay before sending ACK (allows piggybacking)

T3	Inactive Link Timer	180000 ms (3 min)	Detect dead link when idle

Counter	Name	Default	Purpose
N2	Retry Limit	10	Max retransmit attempts
K	Max Outstanding Frames	4	Sliding window size (max 7 for mod-8)

T1 should be adaptive. A simple approach: T1 = 2 * SRTT + 4 * RTTVAR, where SRTT is the smoothed round-trip time measured from I-frame send to ACK receipt. Initial value = 3000 ms. Minimum 500 ms, maximum 30000 ms. For practical purposes at 1200 baud through digipeaters, values of 5000–15000 ms are common.

4.7 Parameter Negotiation (XID)

AX.25 v2.2 defines XID frames for negotiating link parameters (PACLEN, window size, etc.). For this initial implementation, XID is not required. If an XID frame is received, respond with XID containing our default parameters, or ignore it. SABME (extended mode) is not initiated by this app; if a remote station sends SABME, respond with DM to fall back to v2.0.

5. Terminal Emulation Layer

5.1 Terminal Display

The terminal is the primary user interface during an active connection. It displays received data as scrolling text with a fixed-width font, and provides an input field for composing and sending text.

5.1.1 Display Requirements

- Full-screen terminal view. The terminal text area occupies all available vertical space above the input field.
- Monospaced font. Default: Menlo. Configurable size (default 14pt, range 10–24pt).
- Character encoding: ISO-8859-1 (Latin-1). This is the standard character set for packet radio nodes. The app must filter outbound text to this character set, silently dropping unsupported characters (emojis, non-Latin scripts, smart quotes).
- Color themes: At least 4 retro-inspired themes (e.g., green-on-black, amber-on-black, white-on-blue, light mode). Implemented as SwiftUI environment values.
- Auto-scrolling: The terminal always scrolls to the bottom when new data arrives, unless the user has manually scrolled up. Scrolling back down re-enables auto-scroll.
- Scroll buffer: At least 10,000 lines. Older lines are discarded on a FIFO basis.
- Clear terminal: Accessible from the overflow menu. Clears only the display, not the session state.

5.1.2 Data Flow

Received I-frame payloads arrive as raw bytes from the AX25Session. The terminal converts them to String using ISO-8859-1 (Latin-1) encoding and appends to the display buffer. Common control characters:

- CR (0x0D): Carriage return. Move to start of line.
- LF (0x0A): Line feed. Move down one line.
- BS (0x08): Backspace. Delete previous character.
- BEL (0x07): Bell. Optionally play a system sound or haptic.
- Other control characters (0x00–0x06, 0x0E–0x1F): Discard or display as placeholder.

The terminal should handle both CR+LF and bare CR or bare LF as line endings, as different packet nodes use different conventions.

5.2 Input Handling

- Single-line text input field at the bottom of the screen, above the keyboard.
- Send button (or Return key) transmits the current input text as an I-frame payload, appending CR.
- Voice dictation: Standard iOS dictation via the microphone button on the keyboard. The dictated text appears in the input field for review before sending.
- Auto-capitalization: On by default. Capitalizes the first letter of sentences. Toggled by long-pressing a keyboard settings button.
- Auto-correction: Off by default (packet node commands are not natural language). Toggled by long-pressing a keyboard settings button.

5.3 YAPP File Transfer Protocol

YAPP (Yet Another Packet Protocol) is a binary file transfer protocol used by packet radio BBS systems. The app must support both sending and receiving files via YAPP.

5.3.1 YAPP Packet Types

Byte 0	Byte 1	Name	Description
`ENQ`	`0x01`	SI	Send Init: sender ready to begin
`ACK`	`0x01`	RR	Receive Ready: receiver ready
`ACK`	`0x02`	RF	Receive File: receiver ready for file data
`ACK`	`ACK`	RT	Receive TPK: accept extended YAPP with checksum
`SOH`	`len`	HD	Header: filename + NUL + size + NUL [+ date + time + NUL]
`STX`	`len`	DT	Data: file data bytes (len=0 means 256 bytes)
`ETX`	`0x01`	EF	End of File
`EOT`	`0x01`	ET	End of Transfer
`NAK`	`len`	NR	Not Ready: reject file
`CAN`	`CAN`	CN	Cancel: abort transfer

5.3.2 YAPP Receive State Machine

13. In terminal, detect incoming ENQ 0x01 (SI) from remote. Auto-switch to YAPP receive mode.
14. Send ACK 0x01 (RR) to indicate readiness.
15. Receive SOH header. Parse filename (up to NUL) and file size (ASCII digits up to NUL). Optionally parse date/time if present.
16. Send ACK 0x02 (RF) to accept file, or NAK to reject.
17. Receive STX data packets. Accumulate file data. Display progress in terminal.
18. Receive ETX 0x01 (End of File). Optionally send ACK 0x03 (AF) to acknowledge.
19. Receive EOT 0x01 (End of Transfer). Send ACK 0x04 (AT). Save file using iOS share sheet or Files.

5.3.3 YAPP Send

Initiated from the terminal overflow menu ("Send File..."). The user picks a file via UIDocumentPickerViewController. The app then runs the YAPP send state machine, sending SI, waiting for RR, sending HD, waiting for RF/RT, sending DT packets, then EF and ET.

6. Monitor Mode

The monitor provides a real-time view of all AX.25 traffic on the frequency. It decodes every frame received from the TNC and presents it in a structured, readable format.

6.1 Frame Display

Each received frame is displayed as a card or row showing:

- Timestamp (optional, togglable)
- Source callsign-SSID
- Destination callsign-SSID
- Via path (digipeater addresses, with * marking repeated hops)
- Frame type (I, RR, RNR, REJ, SABM, UA, DISC, DM, UI, FRMR, etc.)
- P/F bit status
- Sequence numbers (N(S), N(R)) for I and S frames
- Information field as ASCII text (for I and UI frames)
- Hex dump option for non-printable data

6.2 Filtering

- Show/hide supervisory frames (RR, RNR, REJ): Off by default reduces clutter.
- Show only UI frames: Useful for seeing beacons, APRS, and node advertisements.
- Text search: Filter by callsign or content substring.

6.3 Log Export

The complete capture (unfiltered) can be exported to a text file via the share sheet. Format: one frame per line with timestamp, addresses, type, and decoded content.

6.4 Mutual Exclusion with Terminal

Monitoring and active terminal connections are mutually exclusive at the TNC level. Starting a monitor session disconnects any active terminal session. Connecting to a node stops monitoring. This is because the TNC delivers all frames on the channel to the host, and during a connection, the AX.25 engine must process all frames to maintain protocol state. A future enhancement could allow simultaneous monitoring during a connection by feeding all frames through both the AX25Engine and the MonitorEngine.

7. Node and Connection Management

7.1 Node Database

The app maintains a database of known packet nodes. Each node record contains:

- Callsign-SSID (e.g., "W2ASM-1")
- Frequency in MHz (e.g., 145.050)
- Description / location (free text)
- Baud rate (1200 or 9600)
- Created / last-connected timestamps

Nodes are displayed in a scrollable list on the main screen. Tapping a node opens the connection sheet.

7.2 Digipeater Path Management

Digipeaters are stored as a separate entity. Each digipeater has:

- Callsign-SSID
- Optional alias (e.g., "WIDE1-1")
- Notes

When connecting to a node, the user can select up to 2 digipeaters from a picker. The digipeater callsigns are inserted into the AX.25 address field of all outgoing frames.

7.3 Location-Aware Smart Paths

When the user successfully connects to a node, the app records the digipeater path (or lack thereof) along with the current location, quantized to 0.01 degrees of latitude and longitude (approximately 1 km). The next time the user opens the connection sheet for the same node from a location within the same grid cell, the previously successful path is pre-selected.

Implementation: store tuples of (nodeID, quantizedLat, quantizedLon, digipeaterPath) in CoreData. On connection sheet open, query for the current node and current quantized location. If a match exists, pre-populate the digipeater picker.

8. Data Persistence and Sync

Nodes, digipeaters, and smart-path records are stored in CoreData with a CloudKit backing store, enabling seamless iCloud sync across the user's devices.

- CoreData stack: NSPersistentCloudKitContainer
- Entities: Node, Digipeater, SmartPath, Settings
- Sync conflict resolution: Last-writer-wins (default CloudKit behavior)
- The user's callsign is stored in UserDefaults (not synced, as it may differ per device in edge cases, but typically the same).
- Terminal scroll-back buffer and monitor capture logs are ephemeral (in-memory only, not persisted).

9. Radio Integration

Some radios with built-in Bluetooth TNCs support frequency control via vendor-specific BLE commands. When connected to such a radio, the app can automatically set the frequency before connecting to a node and restore the original frequency after disconnecting.

9.1 Supported Radios for Frequency Control

- BTECH UV-PRO, UV-50PRO
- VERO VR-N76, VR-N7600
- Radiodity GA-5WB, DB50-B
- Kenwood TH-D74, TH-D75 (via B.B. Link BLE adapter)
- PicoAPRS V4

Each radio uses vendor-specific BLE service UUIDs and characteristic protocols for frequency control. The app must implement per-vendor adapters. Details of each vendor protocol are outside the scope of this core specification and should be documented in separate vendor integration appendices.

9.2 Frequency Set/Restore Workflow

20. When the user taps Connect on a node that has a frequency specified:
21. If the TNC is a supported radio, read its current frequency and save it.
22. Set the radio to the node's frequency via vendor BLE commands.
23. Wait for confirmation. Then send the SABM via KISS.
24. On disconnect (or connection failure), restore the saved frequency.

10. User Interface Specification

10.1 Tab Structure

The app uses a 3-tab layout:

- **Nodes tab:** List of saved nodes with Add/Edit. Tap a node to open Connection Sheet.
- **Monitor tab:** Start/Stop monitoring, frame display, filters.
- **Settings tab:** Callsign, TNC selection, KISS parameters (TXDelay, Persistence, SlotTime), terminal appearance (theme, font size), About.

10.2 Connection Sheet (Modal)

Presented when the user taps a node. Shows node callsign, frequency, baud rate. Digipeater picker ("via" field) with smart path pre-selection. Connect button. Status indicator during connection attempt. If the radio supports frequency control, shows the HT icon indicator.

10.3 Terminal View

Full-screen after connection is established. Shows terminal text area and input field. Overflow menu provides: Clear Terminal, Send File (YAPP), Disconnect. Connection status indicator (connected/disconnecting). If the connection drops, a banner appears with the reason.

10.4 Accessibility

- Dynamic Type support for all UI text (except the terminal, which uses a fixed monospace font with user-configurable size).
- VoiceOver labels on all interactive elements.
- Sufficient color contrast in all themes (WCAG AA minimum).

11. Error Handling and Edge Cases

11.1 BLE Disconnection

If the BLE connection to the TNC drops unexpectedly, the app transitions to the disconnected state for both the TNC transport and any active AX.25 session. The user is notified. The app does not attempt automatic BLE reconnection (to avoid draining battery), but offers a Reconnect button.

11.2 AX.25 Protocol Errors

- FRMR received: The remote station detected a protocol error. Log the FRMR information field (which contains the rejected control byte, V(S), V(R), and error flags). Disconnect the session and notify the user.
- DM received during connection: The remote station refused or terminated the link. Transition to S1.
- Invalid frame received: Frames with bad addressing (no extension bit set), unknown frame types, or malformed control fields should be silently discarded and logged to the monitor.

11.3 Buffer Overflow

If the remote station sends data faster than the terminal can display or the user can read, the app must still accept and acknowledge I-frames (to avoid protocol deadlock). The scroll buffer is bounded at 10,000 lines; excess is trimmed from the top.

11.4 Background Execution

iOS suspends apps when backgrounded. An active AX.25 connection will effectively freeze. On return to foreground, the connection may have been dropped by the remote station (T1 timeouts). The app should detect this by sending an RR with P=1 (link check) and handling the response or timeout. BLE connections persist in the background for a limited time but KISS data flow stops. The app should warn the user if they background during an active connection.

12. Testing Strategy

12.1 Unit Tests

- AX25Frame: Encode/decode round-trip for all frame types (I, RR, RNR, REJ, SABM, UA, DISC, DM, UI, FRMR). Verify address encoding with various callsigns (1–6 chars, with SSIDs, with digipeaters).
- KISSCodec: Escape/unescape round-trip. Frame assembly from fragmented input. Back-to-back FEND handling. Embedded FEND/FESC in payload.
- AX25StateMachine: State transitions for all events in all states. Timer expiry. N2 exhaustion. Sequence number wraparound (modulo 8). Window full condition.
- YAPP: Parse and generate all YAPP packet types. Full send/receive state machine with mock transport.
- AFSK Modulator/Demodulator: Generate known AFSK audio from a known bit pattern, then demodulate it and verify bit-perfect recovery.

12.2 Integration Tests

- Connect to a Dire Wolf instance over TCP KISS. Perform a full SABM/UA/I-frame exchange.
- Connect to a Mobilinkd TNC4 via BLE. Monitor traffic on a local frequency.
- Full YAPP file transfer to a BPQ32 BBS running on a local Raspberry Pi.

12.3 Simulated TNC

Build a SimulatedKISSTransport conforming to the KISSTransport protocol. It contains a simple AX.25 responder (accepts SABM, echoes I-frames, responds to DISC) for development and testing without radio hardware.

13. Appendix A: KISS Protocol Byte Reference

Hex	Constant	Usage
`0xC0`	FEND	Frame delimiter
`0xDB`	FESC	Escape character
`0xDC`	TFEND	Escaped FEND (after FESC)
`0xDD`	TFESC	Escaped FESC (after FESC)
`0x00`	CMD_DATA	Data frame type byte
`0x01`	CMD_TXDELAY	Set TX delay
`0x02`	CMD_P	Set persistence
`0x03`	CMD_SLOTTIME	Set slot time
`0x04`	CMD_TXTAIL	Set TX tail
`0x05`	CMD_DUPLEX	Set full/half duplex
`0x06`	CMD_SETHW	Set hardware (vendor-specific)
`0xFF`	CMD_RETURN	Exit KISS mode

14. Appendix B: AX.25 Frame Type Reference

Control byte encoding for AX.25 v2.0 modulo-8:

Frame	Type	Control Byte	Has Info Field
I	I	`NR P NS 0`	Yes (user data)
RR	S	`NR P/F 0001`	No
RNR	S	`NR P/F 0101`	No
REJ	S	`NR P/F 1001`	No
SABM	U	`0010 P 1111`	No
DISC	U	`0100 P 0011`	No
UA	U	`0110 F 0011`	No
DM	U	`0001 F 1111`	No
UI	U	`0000 P 0011`	Yes (unconnected data)
FRMR	U	`1000 F 0111`	Yes (3 bytes error info)

15. Appendix C: BLE KISS UUID Reference

Name	UUID
KTS_SERVICE_UUID	`00000001-ba2a-46c9-ae49-01b0961f68bb`
KTS_TX_CHAR_UUID	`00000002-ba2a-46c9-ae49-01b0961f68bb`
KTS_RX_CHAR_UUID	`00000003-ba2a-46c9-ae49-01b0961f68bb`

TX = host to TNC (write without response). RX = TNC to host (notify). The app subscribes to RX notifications to receive frames and writes to TX to transmit frames.

DNS-SD service type for TCP KISS discovery: _kiss-tnc._tcp

16. Appendix D: AFSK Modem DSP Parameters

Parameter	Value
Mark Frequency	1200 Hz
Space Frequency	2200 Hz
Baud Rate	1200 baud
Symbol Period	833.33 µs
Sample Rate	22050 Hz (or 44100 Hz)
Samples per Symbol (22050)	18.375
Samples per Symbol (44100)	36.75
HDLC Flag	0x7E (01111110)
FCS Polynomial	CRC-CCITT, 0x8408 (reflected)
FCS Good Residual	0xF0B8
Encoding	NRZI (0=transition, 1=no transition)
Bit Order	LSB first
Preamble	Minimum 300 ms of flags (36 flags at 1200 baud)
TX Delay (configurable)	Default 500 ms (50 * 10 ms units)
Demod Bandpass Filter	900–2500 Hz, 4th order Butterworth

— End of Specification —